

Sistema de Verificação de Dados

Documentação Técnica do Módulo de Pré-processamento

Instituto de Ciências Matemáticas e de Computação

20 de julho de 2026

Sumário

1	Introdução	3
1.1	Objetivo	3
2	Estrutura de Dados	3
2.1	Conjuntos Principais	3
2.1.1	Conjunto de Turmas/Disciplinas (T)	3
2.1.2	Conjunto de Salas (S)	3
2.1.3	Conjunto de Aulas (A)	3
2.1.4	Conjunto de Cursos (C)	3
2.2	Parâmetros de Entrada	4
2.2.1	Capacidade das Salas	4
2.2.2	Tamanho das Turmas	4
2.2.3	Horários das Aulas	4
2.2.4	Identificadores Binários	4
2.3	Parâmetros Calculados	4
2.3.1	Compatibilidade Aula-Sala	4
2.3.2	Conflito de Horário	5
2.3.3	Utilização de Espaço	5
2.3.4	Distância entre Salas	5
2.3.5	Matriz Turma-Curso	5
3	Estruturas de Pré-processamento	5
3.1	Agrupamento de Aulas por Turma	5
3.1.1	Aulas por Turma com Horário Definido	5
3.1.2	Aulas Consecutivas	5
3.2	Agrupamento de Aulas por Curso	6
3.3	Intervalos de Horário Padrão	6
4	Grafos de Conflito	6
4.1	Construção do Grafo	6
4.2	Componentes Conexas	6
5	Algoritmos de Verificação	6
5.1	Verificação de Fixações	6
5.2	Verificação de Disponibilidade (Emparelhamento Perfeito)	6

6	Tratamento de Casos Especiais	8
6.1	Salas Duplas de Laboratório	8
6.2	Proibição de Salas	8
7	Mensagens de Erro	8
7.1	Erros de Capacidade	8
7.2	Erros de Fixação	8
7.3	Erros de Disponibilidade	8
7.4	Erros de Digitação (Typos)	9
8	Complexidade Computacional	9
8.1	Análise de Complexidade	9
9	Saída do Sistema	9
9.1	Caso de Sucesso	9
9.2	Caso de Falha	9
10	Considerações de Implementação	10
10.1	Estruturas de Dados Utilizadas	10
10.2	Dependências	10
11	Conclusão	10

1 Introdução

Este documento descreve o módulo de verificação de dados utilizado no sistema de alocação de salas do ICMC-USP. O módulo `verificar_dados.py` é responsável por validar a consistência e viabilidade dos dados de entrada antes da execução do modelo de otimização.

1.1 Objetivo

O objetivo principal deste módulo é identificar conflitos e inconsistências nos dados de entrada que impossibilitariam uma solução viável do problema de alocação de salas, incluindo:

- Conflitos de horário entre aulas fixadas na mesma sala
- Insuficiência de salas para grupos de aulas conflitantes
- Incompatibilidade entre capacidade de salas e número de alunos
- Conflitos entre aulas de laboratório e salas disponíveis
- Fixações inválidas de salas

2 Estrutura de Dados

2.1 Conjuntos Principais

2.1.1 Conjunto de Turmas/Disciplinas (T)

$$T = \{0, 1, 2, \dots, |T| - 1\} \quad (1)$$

Representa o conjunto de todas as turmas/disciplinas a serem alocadas, indexadas de 0 até $|T| - 1$.

2.1.2 Conjunto de Salas (S)

$$S = \{0, 1, 2, \dots, |S| - 1\} \quad (2)$$

Representa o conjunto de todas as salas disponíveis no instituto.

2.1.3 Conjunto de Aulas (A)

$$A = \{0, 1, 2, \dots, |A| - 1\} \quad (3)$$

Representa o conjunto de todas as aulas individuais, onde $|A| = |T| \times n_{horarios}$, sendo $n_{horarios}$ o número de colunas de horário (tipicamente 4).

2.1.4 Conjunto de Cursos (C)

$$C = \{\text{BMACC}, \text{BMA}, \text{LMA}, \text{MAT-NG}, \text{BECD}, \text{BCC}, \text{BSI}, \text{BCDados}\} \quad (4)$$

Representa os currículos oferecidos pelo ICMC.

2.2 Parâmetros de Entrada

2.2.1 Capacidade das Salas

$$\text{cap}_s \in \mathbb{Z}^+, \quad \forall s \in S \quad (5)$$

Número de lugares disponíveis na sala s .

2.2.2 Tamanho das Turmas

$$\text{tam}_t \in \mathbb{Z}^+, \quad \forall t \in T \quad (6)$$

Número de alunos inscritos na turma/disciplina t .

2.2.3 Horários das Aulas

Para cada aula $a \in A$:

$$\text{dia}_a \in \{\text{Seg, Ter, Qua, Qui, Sex}\} \quad (7)$$

$$\text{start}_a \in \mathbb{R}^+ \cup \{0\} \quad (8)$$

$$\text{end}_a \in \mathbb{R}^+ \cup \{0\} \quad (9)$$

Onde start_a e end_a são representados em formato decimal de horas. Por exemplo, 14:20 é representado como:

$$14 : 20 \rightarrow 14 + \frac{20}{60} = 14.33 \quad (10)$$

2.2.4 Identificadores Binários

Sala de Laboratório:

$$\sigma_s = \begin{cases} 1, & \text{se a sala } s \text{ é laboratório} \\ 0, & \text{caso contrário} \end{cases} \quad (11)$$

Requer Laboratório:

$$\tau_t = \begin{cases} 1, & \text{se a turma } t \text{ requer laboratório} \\ 0, & \text{caso contrário} \end{cases} \quad (12)$$

Aula é de Laboratório:

$$\text{lab_tal}_a = \begin{cases} 1, & \text{se a aula } a \text{ é de laboratório} \\ 0, & \text{caso contrário} \end{cases} \quad (13)$$

2.3 Parâmetros Calculados

2.3.1 Compatibilidade Aula-Sala

$$\eta_{a,s} = \begin{cases} 1, & \text{se } \text{tam}_{a \bmod |T|} \leq \text{cap}_s \\ 0, & \text{caso contrário} \end{cases} \quad (14)$$

A operação $a \bmod |T|$ converte o índice da aula para o índice da turma correspondente.

2.3.2 Conflito de Horário

Duas aulas a e a' têm conflito de horário se:

$$\theta_{a,a'} = \begin{cases} 1, & \text{se } \text{dia}_a = \text{dia}_{a'} \text{ e } (\text{start}_a < \text{end}_{a'} \text{ e } \text{start}_{a'} < \text{end}_a) \\ 0, & \text{caso contrário} \end{cases} \quad (15)$$

2.3.3 Utilização de Espaço

$$\text{uso}_{a,s} = 100 \times \left(1 - \frac{\text{tam}_a \bmod |T|}{\text{cap}_s} \right) \quad (16)$$

Representa a porcentagem de espaço não utilizado se a aula a for alocada na sala s .

2.3.4 Distância entre Salas

$$\text{dis}_{s,s'} \in \mathbb{R}^+ \quad (17)$$

Distância arbitrária entre as salas s e s' , considerando proximidade física entre blocos.

2.3.5 Matriz Turma-Curso

$$Y_{t,c} = \begin{cases} 1, & \text{se a turma } t \text{ é ministrada para o curso } c \\ 0, & \text{caso contrário} \end{cases} \quad (18)$$

3 Estruturas de Pré-processamento

3.1 Agrupamento de Aulas por Turma

3.1.1 Aulas por Turma com Horário Definido

$$A_{tt}(t) = \{a \in A_t(t) : \text{start}_a \neq 0\} \quad (19)$$

onde $A_t(t)$ representa todas as aulas da turma t :

$$A_t(t) = \left\{ t + i \cdot |T| : i \in \left\{ 0, 1, \dots, \frac{|A|}{|T|} - 1 \right\} \right\} \quad (20)$$

3.1.2 Aulas Consecutivas

Duas aulas $a_1, a_2 \in A_t(t)$ são consideradas consecutivas se:

$$\text{dia}_{a_1} = \text{dia}_{a_2} \quad (21)$$

$$\text{end}_{a_1} + \Delta t \geq \text{start}_{a_2} \quad (22)$$

onde $\Delta t = 2 + \frac{10}{60} + \frac{20}{60} = 2.5$ horas é o intervalo máximo entre aulas consecutivas.

O conjunto de todas as aulas consecutivas é:

$$A_s = \{(a_1, a_2) : a_1, a_2 \text{ são consecutivas}\} \quad (23)$$

3.2 Agrupamento de Aulas por Curso

Para cada curso $c \in C$:

$$A_c(c) = \{a \in A : Y_{a \bmod |T|,c} = 1\} \quad (24)$$

3.3 Intervalos de Horário Padrão

O sistema utiliza os seguintes intervalos para agrupar aulas:

$$I_1 = [8 : 00, 9 : 50] \quad (25)$$

$$I_2 = [10 : 00, 13 : 00] \quad (26)$$

$$I_3 = [13 : 00, 16 : 25] \quad (27)$$

$$I_4 = [16 : 00, 18 : 00] \quad (28)$$

$$I_5 = [19 : 00, 20 : 50] \quad (29)$$

$$I_6 = [21 : 00, 22 : 50] \quad (30)$$

Uma aula a pertence ao intervalo I_i se:

$$\text{start}_a \geq I_i^{\text{inicio}} \wedge \text{end}_a \leq I_i^{\text{fim}} \quad (31)$$

Aulas que não se encaixam em exatamente um intervalo são consideradas problemáticas.

4 Grafos de Conflito

4.1 Construção do Grafo

O sistema constrói um grafo não-direcionado $G = (V, E)$ onde:

- $V = A$ (cada vértice representa uma aula)
- $(a, a') \in E \Leftrightarrow \theta_{a,a'} = 1$ (existe aresta se há conflito)

4.2 Componentes Conexas

Cada componente conexa de G representa um grupo de aulas que possuem conflitos diretos ou indiretos. O conjunto de grupos de conflito é:

$$\mathcal{G} = \{G_1, G_2, \dots, G_k\} \quad (32)$$

onde cada $G_i \subseteq A$ é um grupo maximal de aulas conflitantes.

5 Algoritmos de Verificação

5.1 Verificação de Fixações

5.2 Verificação de Disponibilidade (Emparelhamento Perfeito)

Para verificar se um grupo de conflito $G_i \in \mathcal{G}$ pode ser alocado fisicamente nas salas disponíveis, o sistema abandona abordagens heurísticas de contagem e resolve um problema

Algorithm 1 Verificação de Fixações de Salas

```
1: for  $a \in A$  do
2:   if  $a$  tem sala fixada  $s_{fix}$  then
3:     if  $lab\_tal_a = 1$  e  $\sigma_{s_{fix}} = 0$  then
4:       return ERRO: Lab fixado em sala comum
5:     end if
6:     if  $lab\_tal_a = 0$  e  $\sigma_{s_{fix}} = 1$  then
7:       return ERRO: Aula comum fixada em lab
8:     end if
9:     if  $\eta_{a,s_{fix}} = 0$  then
10:      return ERRO: Aula não cabe na sala fixada
11:    end if
12:    for  $a' \in A : \theta_{a,a'} = 1$  do
13:      if  $a'$  tem sala fixada  $s'_{fix}$  e  $s_{fix} = s'_{fix}$  then
14:        return ERRO: Conflito de fixações
15:      end if
16:    end for
17:  end if
18: end for
```

de Emparelhamento Perfeito Bipartido (Bipartite Matching). Esta abordagem garante que existe uma atribuição um-para-um entre aulas e salas, respeitando simultaneamente todas as restrições individuais de capacidade, proibições e fixações ($\eta_{a,s}$).

Algorithm 2 Algoritmo de Emparelhamento (Kuhn)

```
1: function EMPARELHAMENTO( $G_i, S_{permitidas}$ )
2:    $ocupada\_por \leftarrow$  vetor mapeando  $s \in S_{permitidas}$  para -1
3:    $alocadas \leftarrow 0$ 
4:   for cada aula  $a \in G_i$  do
5:      $visitados \leftarrow$  vetor mapeando  $s \in S_{permitidas}$  para Falso
6:     if TENTARALOCAR( $a, visitados, ocupada\_por$ ) then
7:        $alocadas \leftarrow alocadas + 1$ 
8:     end if
9:   end for
10:  if  $alocadas < |G_i|$  then
11:    return ERRO: Insuficiência de salas ou gargalo físico (Falta de espaço)
12:  end if
13:  return SUCESSO
14: end function
```

A função auxiliar TENTARALOCAR utiliza Busca em Profundidade (DFS) iterando sobre as salas para realocar aulas concorrentes caso encontre gargalos (caminhos aumentantes). Este método é aplicado independentemente para o conjunto de aulas normais (utilizando $S \setminus S_{labs}$) e aulas de laboratório (utilizando apenas S_{labs}).

6 Tratamento de Casos Especiais

6.1 Salas Duplas de Laboratório

Para laboratórios que podem ser usados em conjunto:

- Se uma aula é fixada em 6-303/6-304, as salas 6-303 e 6-304 não podem ser usadas individualmente
- Se uma aula é fixada em 6-305/6-306, as salas 6-305 e 6-306 não podem ser usadas individualmente

6.2 Proibição de Salas

O sistema permite proibir salas específicas para determinadas aulas através do parâmetro:

$$\text{proibida}_a \subseteq S \quad (33)$$

Para toda sala $s \in \text{proibida}_a$:

$$\eta_{a,s} \leftarrow 0 \quad (34)$$

7 Mensagens de Erro

O sistema produz mensagens de erro específicas para cada tipo de inconsistência:

7.1 Erros de Capacidade

- **E1:** Não há sala capaz de comportar a disciplina t
- **E2:** Aulas fixadas em sala onde não cabem

7.2 Erros de Fixação

- **E3:** Aula de laboratório fixada em sala comum
- **E4:** Aula comum fixada em laboratório
- **E5:** Aulas conflitantes fixadas na mesma sala

7.3 Erros de Disponibilidade

- **E6:** Há muitas aulas com conflito de horário no grupo G_i
- **E7:** Insuficiência de laboratórios para grupo L_i

7.4 Erros de Digitação (Typos)

Para evitar a quebra abrupta do sistema por falhas de preenchimento (dados ausentes ou digitados incorretamente na planilha):

- **E8:** Erro de Digitação (Cursos): O curso especificado não existe na lista de currículos válidos.
- **E9:** Erro de Digitação (Sala Fixada): A sala definida não existe na aba de Salas.
- **E10:** Erro de Digitação (Sala Proibida): A sala proibida não existe na aba de Salas.

8 Complexidade Computacional

8.1 Análise de Complexidade

- **Leitura e processamento de dados:** $O(|T| \times n_{horarios})$
- **Construção da matriz $\theta_{a,a'}$:** $O(|A|^2)$
- **Identificação de grupos de conflito:** $O(|A|^2)$
- **Verificação de fixações:** $O(|A| \times |S|)$
- **Verificação de disponibilidade (Emparelhamento Bipartido):** $O(|\mathcal{G}| \times |A| \times |S|)$ no pior caso usando o Algoritmo de Kuhn.

Complexidade total: $O(|A|^2 + |\mathcal{G}| \times |A| \times |S|)$

Na prática, como $|A| = O(|T|)$ e $|\mathcal{G}| \ll |A|$, a complexidade é dominada por $O(|T|^2)$.

9 Saída do Sistema

9.1 Caso de Sucesso

Se todas as verificações são bem-sucedidas, o sistema imprime:

"Aparentemente, não há nenhum conflito de horário, e há salas disponíveis em todos os horários necessários."

9.2 Caso de Falha

Em caso de erro, o sistema:

1. Imprime a mensagem de erro específica
2. Lista as disciplinas/aulas envolvidas no conflito
3. Termina a execução com código de erro

10 Considerações de Implementação

10.1 Estruturas de Dados Utilizadas

- **DataFrames pandas:** Para armazenamento e manipulação dos dados de entrada
- **Listas Python:** Para conjuntos ordenados (turmas, aulas, salas)
- **Dicionários:** Para matrizes esparsas e grafos customizados (η, θ, Y)

10.2 Dependências

```
1 import pandas as pd
2 import sys
3 import traceback
```

11 Conclusão

O módulo de verificação de dados é essencial para garantir que o modelo de otimização receberá dados consistentes e viáveis. Através de uma série de verificações sistemáticas, o sistema identifica proativamente conflitos e inconsistências que impossibilitariam uma alocação válida de salas.

A abordagem baseada em grafos de conflito permite uma análise eficiente e completa das restrições, enquanto a categorização por capacidade garante que as salas sejam utilizadas de forma otimizada.